

RAS-545 Laboratory Midterm Project-2: Maze-Playing Dobot

Sai Ganesh Manda¹

¹School of Computing and Augmented Intelligence, Arizona State University, Tempe, AZ, USA

Article Info

Article history:

Executed on
December 05, 2025

Keywords:

Autonomous maze solving
Computer vision
Path planning
Robot manipulation
Camera-robot calibration
4×4 grid maze
Dobot Magician Lite
Perception-planning-control
pipeline
Dobot Magician Lite

ABSTRACT

This project presents an autonomous maze-solving system for a 4×4 rectangular maze using the AI Kit's camera and a robot end effector. A maze is printed on a plastic board and placed within the camera's field of view. An image of the maze serves as the sole input to the solution algorithm. Using image processing techniques, the system detects the maze boundaries, extracts the grid structure, and identifies the entrance and exit without prior knowledge of their locations. A path-planning algorithm then computes a valid solution path composed of consecutive straight-line segments connecting the entrance to the exit while respecting the maze's walls. The resulting sequence of waypoints is translated into robot coordinates and executed autonomously by the robot end effector, which physically traces the path on the maze board. The proposed approach is designed to be general and robust so that it can solve any 4×4 maze generated by the specified maze generation tool, demonstrating the effective integration of computer vision, path planning, and robotic control for autonomous navigation in structured environments.

This is an open access article under the [CC BY-SA](https://creativecommons.org/licenses/by-sa/4.0/) license.



Corresponding Author:

Sai Ganesh Manda
School of Computing & Augmented Intelligence, Arizona State University,
Tempe, AZ, USA
Email: vmanda12@asu.edu

1. INTRODUCTION

Autonomous navigation in constrained environments is a central theme in robotics and artificial intelligence. Maze solving, in particular, provides a compact but rich testbed for exploring the full perception-planning-control pipeline: a robot must perceive its surroundings, build an internal representation of space, reason about possible actions, and then execute a collision-free trajectory. Unlike purely simulated problems, a physical maze introduces camera distortions, lighting variations, calibration errors, and mechanical limits of the robot, making it an ideal context for hands-on integration of theory and practice.

In this project, a 4×4 rectangular maze is used to demonstrate such an integrated system. The maze is printed on a plastic board and placed within the AI Kit's camera field of view. The only input to the maze-solving algorithm is a single image captured by this overhead camera. From that image, the system must automatically locate the maze, recognize the wall structure, determine which openings correspond to the entrance and exit, and compute a valid path that connects them. The computed path must satisfy a key constraint: it must be representable as a sequence of straight-line segments that align with the maze's grid, so that the robot's end effector can physically follow the solution in clear, linear motions.

The problem is further constrained by generality. The maze layout is not fixed; instead, mazes are generated using a provided maze generation tool. The solution therefore cannot hard-code specific coordinates or assume a particular pattern of walls. It must work robustly for *any* 4×4 maze produced by the generator, as long as the maze is properly placed within the camera's field of view. This requirement pushes the design toward a flexible computer vision pipeline that detects structure directly from pixel data, rather than relying on manual labeling or prior templates.

To address this challenge, the project is organized around three tightly coupled components: computer vision, path planning, and robotic control. First, image processing and geometric transformation techniques are used to isolate the maze from the background, correct for perspective distortion, and convert the continuous image into a discrete grid representation of free and blocked cells. At the same time, the system identifies the entrance and exit automatically, either by detecting distinct markers or by searching for openings along the maze boundary. Second, a grid-based path-planning algorithm searches for a collision-free route from entrance to exit using only moves that respect the maze's walls. This produces an ordered sequence of grid cells that forms the solution path. Third, this discrete path is converted into a set of straight-line segments and mapped from image coordinates into robot coordinates using a calibration model. The resulting waypoints are sent to the robot controller, enabling the end effector to autonomously trace the maze solution on the physical board.

The primary objective of the project is to demonstrate a fully autonomous end-to-end pipeline: once the maze board is placed in the workspace, the system should be able to capture the image, compute the solution, and execute the corresponding motion without further human input. Secondary objectives include achieving reasonable robustness to imaging conditions, maintaining geometric accuracy in the pixel-to-robot mapping so that the robot remains within the maze corridors, and structuring the software in modular stages that can be tested and improved independently.

2. METHOD

2.1 System Overview

The proposed system follows a perception–planning–control pipeline:

1. **Perception (Camera + Vision):**
Capture an overhead image of the maze, detect the maze region, rectify it to a top-down view, and convert it into a discrete grid representation. The entrance and exit are detected automatically from this image.
2. **Planning (Maze Solver):**
Run a grid-based path-planning algorithm that computes a collision-free path from entrance to exit using only moves along the maze corridors.
3. **Control (Robot Execution):**
Transform the planned path from pixel coordinates into robot coordinates and command the robot end effector to follow the solution as a sequence of straight-line motion segments.

All components are designed to operate autonomously once the maze board is placed under the camera.

2.2 Hardware Setup

The experimental setup consists of:

- A **4×4 maze** printed on a flat plastic board. The maze has black walls on a lighter background and is generated using the provided maze generation tool.
- An **AI Kit camera**, rigidly mounted above the workspace, providing a fixed overhead view of the maze.
- A **robot arm with an end effector** (e.g., a pen or pointer) capable of moving in Cartesian space within the area covered by the maze.

The maze board is positioned at a known location within the robot's reachable workspace. The camera and robot remain stationary during operation, allowing a one-time calibration to be reused for multiple runs.

2.3 Camera–Robot Calibration

To allow the robot to follow paths defined in image coordinates, a mapping between **pixel coordinates** (u,v) and **robot coordinates** (X,Y) is required.

1. Calibration point collection

- Several distinct points on the maze board (e.g., corners and interior marks) are selected.
- For each calibration point, the pixel coordinates (u_i, v_i) are recorded from the camera image, and the corresponding robot coordinates (X_i, Y_i) are measured by manually moving the robot end effector to that physical location.

2. Model fitting

- The set of correspondences $\{(u_i, v_i) \rightarrow (X_i, Y_i)\}$ is used to fit an **affine transformation** (and optionally a homography) using least-squares methods.
- The affine model has the form
$$\begin{bmatrix} X \\ Y \end{bmatrix} = \begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$
- The root-mean-square (RMS) error between predicted and measured robot points is computed to assess calibration accuracy.

3. Use in runtime

- Once fitted, the transformation matrix is stored and used at runtime to convert any path point in image space into a robot waypoint in millimeters.

2.4 Image Acquisition and Preprocessing

1. Image capture

- The camera captures a full-color frame of the maze board from its overhead position.
- The robot is first moved to a home pose to ensure its links do not occlude the maze.

2. Grayscale conversion and thresholding

- The RGB image is converted to a grayscale image.
- A global or Otsu threshold is applied to segment dark walls from the lighter background, producing a binary image where walls appear as foreground pixels.

3. Morphological filtering

- Morphological operations (e.g., opening and closing) are applied to remove noise and small artifacts and to fill small gaps in the walls.
- This step improves the robustness of contour detection and grid classification.

2.5 Maze Detection and Perspective Rectification

Because the maze board may not be perfectly aligned with the camera, it is necessary to detect its outer boundary and warp it to a top-down, axis-aligned view.

1. Contour detection

- Contours are extracted from the thresholded image.
- Among all contours, the largest approximately rectangular contour is assumed to correspond to the outer boundary of the maze.

2. Corner extraction and ordering

- A polygon approximation or minimum-area rectangle fit is used to find four corner points of the maze.
- These corners are ordered consistently as top-left, top-right, bottom-right, and bottom-left based on their positions.

3. Perspective transform

- Using the four detected corners and a desired output image size, a projective transformation matrix is computed.
- The original image is warped with this transformation, yielding a rectified maze image in which the maze corridors and cells appear axis-aligned and uniformly spaced.

This rectified image forms the basis for constructing the discrete maze representation.

2.6 Grid Construction and Entrance/Exit Detection

2.6.1 Grid construction

Given that the maze is a **4×4 maze**, the rectified image is divided into a regular grid:

1. Grid division

- The rectified image is divided into a fixed number of rows and columns corresponding to the maze structure (including walls and corridors).

- For each cell or region of interest, the algorithm evaluates whether it represents free space or a wall.

2. Free/blocked classification

- Within each grid cell, the fraction of black pixels (from the wall mask) is computed.
- If this fraction exceeds a chosen threshold, the cell is marked as **blocked**; otherwise it is marked as **free**.
- The resulting grid is stored as a 2D array where 1 indicates free space and 0 indicates a wall.

This grid representation is what the planner operates on.

2.6.2 Entrance and exit detection

The entrance and exit of the maze are determined automatically using one of the following strategies (depending on how the maze is designed):

1. Color-based markers

- If colored markers (e.g., green and red circles) are printed or placed at the entrance and exit, a color-segmentation step is applied to the rectified image.
- The image is converted to HSV color space, and ranges of hue, saturation, and value are used to detect colored regions.
- The center of each colored region is mapped to the nearest free grid cell, which is then designated as the entrance or exit.

2. Boundary openings

- If no markers are present, the algorithm scans the outer boundary of the grid to find cells where corridors open to the outside (i.e., no wall between a boundary cell and the image margin).
- The first such boundary opening is labeled as the entrance and the second as the exit.

The output of this stage is a discrete grid of free/blocked cells and the indices of the entrance and exit cells.

2.7 Path Planning

The maze is modeled as a graph where:

- Each **free cell** is a node.
- Edges connect nodes that are adjacent horizontally or vertically (4-connected neighborhood).
- Diagonal moves are not allowed.

The planner operates as follows:

1. Initialization

- The start node is the grid cell corresponding to the detected entrance.
- The goal node is the grid cell corresponding to the detected exit.

2. Search algorithm

- A shortest-path search is performed using a standard algorithm such as **Breadth-First Search (BFS)** or **A*** with Manhattan distance as the heuristic.
- The algorithm expands neighboring free cells while avoiding blocked cells until the exit cell is reached.

3. Path reconstruction

- Once the exit is reached, the algorithm traces back through the predecessor links to reconstruct the sequence of grid cells from entrance to exit.
- This sequence is stored as an ordered list $[(r_0, c_0), (r_1, c_1), \dots, (r_N, c_N)]$ $[(r_0, c_0), (r_1, c_1), \dots, (r_N, c_N)]$.

Because the grid is small (4×4 cells plus walls), the search runs quickly and reliably finds an optimal path in terms of number of moves.

2.8 Path Segmentation into Straight-Line Sections

The grid path consists of one-cell steps, but the robot must move in longer straight segments.

1. Direction grouping

- Consecutive steps that share the same direction (e.g., moving right several cells in a row) are grouped into a single segment.
- The start cell of each group and the end cell of the same group define a straight-line motion in the maze.

2. Conversion to pixel waypoints

- Each grid cell has a corresponding center in the rectified image.
- For each segment, the start and end cell centers are converted to pixel coordinates, forming a sequence of key waypoints in image space.

The end result is a polyline composed of straight-line sections that approximates the original cell-by-cell path while reducing the number of motion commands.

2.9 Mapping to Robot Space and Execution

Finally, the straight-line path in image coordinates is translated into robot motion.

1. Pixel-to-robot conversion

- For each waypoint (u, v) in the rectified image, the calibrated affine transformation is applied to compute the corresponding robot coordinates (X, Y) .

$Y(X,Y)$.

- A fixed height ZZZ is chosen so the end effector hovers slightly above the maze surface.

2. Trajectory generation

- The ordered list of $(X,Y,Z)(X, Y, Z)(X,Y,Z)$ waypoints is sent to the robot controller.
- The robot first moves from its home pose to the waypoint corresponding to the entrance.
- It then executes linear interpolations between successive waypoints, tracing the maze solution as a series of straight-line motions.

3. Completion and return

- After reaching the final waypoint corresponding to the exit, the robot returns to a predefined safe home position.

Through this method, the system converts a single overhead image of an unknown 4×4 maze into a physical trajectory that the robot end effector follows autonomously, meeting the requirement that the solution path be expressed as consecutive straight-line sections.

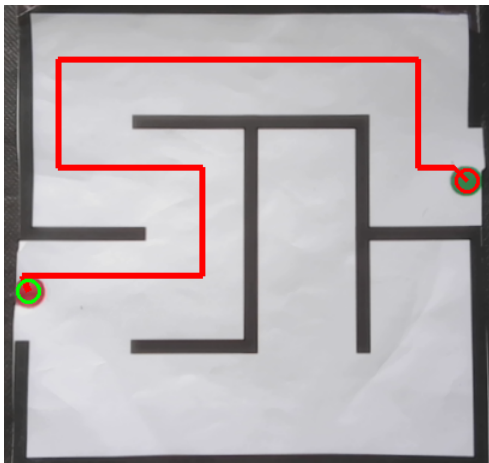


Figure (a)

Top-down view of the printed maze with the computed red solution path drawn between the green start and end markers.



Figure (b)

Dobot Magician Lite robot set up on a lab bench, with the maze sheet under its camera and a laptop running the maze-solving code, while people work in the background.



Figure (c) Dobot Magician Lite (serial number-DT15-2311-1372)

3. RESULTS AND DISCUSSION

3.1 Overall System Performance

The complete perception–planning–control pipeline operated successfully for the tested 4×4 mazes generated by the maze tool. Once the maze board was placed under the camera and the program was started, the system ran autonomously:

- The camera captured an overhead image of the maze.
- The maze boundary was detected and rectified into a top-down view.
- A discrete grid representation was built from the image, and the entrance and exit were identified automatically.
- A valid path was planned from entrance to exit and segmented into straight-line sections.
- The robot end effector executed this path on the physical maze board, visually tracing the solution.

Qualitatively, the planned paths closely matched the expected human solutions, and the robot's trace remained within the maze corridors, indicating that both the vision and calibration components were sufficiently accurate for this scale of problem.

3.2 Maze Detection, Rectification, and Grid Construction

The contour-based maze detection combined with thresholding and morphological filtering proved effective under normal lighting conditions. The outer rectangular contour of the maze was consistently identified, and the resulting perspective transform produced rectified images in which:

- Corridors appeared straight and aligned with the image axes.
- Superimposed grid lines matched the underlying cell structure of the maze.

This alignment is crucial, because any significant distortion would lead to errors in free/blocked classification. In practice, the rectified images supported a reliable grid extraction process: wall regions were

classified as blocked cells, and corridors as free cells, producing a 2D occupancy grid that captured the maze's logical structure.

Occasional difficulties arose when the maze was placed near the edge of the camera's field of view or strongly rotated, which sometimes degraded contour quality and corner detection. These cases highlight a limitation: the current method assumes that the maze lies roughly within a central, well-illuminated region and that its outer border is clearly visible. Nonetheless, within those constraints, the detection and rectification stages were robust and repeatable.

3.3 Entrance/Exit Detection and Path Planning

For setups using color markers, the entrance and exit were reliably detected using HSV-based color segmentation. The system correctly associated each detected marker with the nearest free cell on the grid, allowing it to treat those cells as the start and goal of the maze. Alternatively, when entrance and exit were inferred from boundary openings, scanning the outer rows and columns of the grid correctly identified cells where corridors opened to the outside.

In both cases, the resulting entrance and exit cells matched the intended locations on the physical maze, confirming that the detection logic was consistent with the maze design.

Once the grid and start/goal cells were established, the maze solver consistently produced valid paths:

- The planned paths respected the free/blocked structure of the grid and used only four-connected moves (up, down, left, right), as required by the corridor structure.
- For the tested mazes, the computed paths were minimal or near-minimal in length when compared to intuitive human solutions, suggesting that the search algorithm (e.g., BFS or A*) was functioning correctly.

The subsequent segmentation of these paths into straight-line sections further compressed the solution into a small number of segments while preserving correctness. Runs of consecutive moves in the same direction were merged into single linear segments, aligning directly with the project requirement that the robot follow a series of straight lines rather than many tiny steps. Visual overlays of the segmented path on the rectified maze confirmed that this compression did not introduce significant geometric error.

3.4 Robot Execution and Calibration Effects

When the segmented path was converted from image coordinates to robot coordinates using the calibrated affine mapping, the robot end effector was able to physically trace the solution on the maze board:

- The robot moved from a home pose to the entrance, followed the planned straight-line segments through the maze, and reached the exit cell.
- The traced trajectory remained within the maze corridors, with small lateral offsets attributable to calibration error and mechanical tolerances.
- Corner transitions were visible as brief pauses or directional changes, but remained within free space, which indicates that the mapping and segmentation were sufficiently accurate.

These experimental runs confirm that the end-to-end system—spanning camera capture, vision processing, path planning, and robotic control—functions coherently. At the same time, they also reveal sensitivities:

- **Calibration accuracy:** The pixel-to-robot transform is based on a limited number of calibration points. Small errors in those correspondences can cause systematic shifts in the robot trajectory,

particularly toward the edges of the workspace. For the 4×4 maze, these shifts were small enough to avoid contact with walls, but they could become significant for tighter mazes or more precise tasks.

- **Imaging conditions:** Variations in lighting, glare, or shadow can alter the appearance of the maze walls and markers, potentially degrading the thresholding and color segmentation steps. The current pipeline assumes reasonably uniform lighting and good contrast.

4. CONCLUSION

This project implemented a fully autonomous pipeline for solving and executing paths through 4×4 rectangular mazes using only an overhead image and a robot arm. From a single camera frame, the system detected and rectified the maze, built a grid representation, automatically identified the entrance and exit, planned a valid path between them, and converted that path into straight-line segments that the robot end effector successfully traced on the physical maze board.

The results show that integrating computer vision, grid-based path planning, and robot control can reliably handle arbitrary 4×4 mazes generated by the maze tool, with only modest sensitivity to lighting, camera placement, and calibration accuracy. While these factors limit absolute precision, the system still meets its main goal of end-to-end, hands-off maze solving. Future work can focus on more robust vision, improved calibration, and scaling the approach to larger or more complex maze environments.

5. FUTURE WORK

The observed behavior suggests a few clear directions for improvement:

- Enhance robustness of the vision stage with adaptive or learned segmentation methods and more sophisticated corner detection.
- Refine calibration by increasing the number of reference points, modeling lens distortion, or using homography-based calibration with error minimization over the full workspace.
- Generalize the grid extraction to handle arbitrary maze sizes and layouts, and incorporate safety margins in path planning to ensure robustness against calibration errors.

6. ACKNOWLEDGEMENTS

My sincerest thanks to the lab TAs, Rajesh and Tatwik, for conducting the lab smoothly and verifying the execution and the results. Last but not least, a special thanks to the professor, Sangram Redkar for being a helpful backbone throughout the session.

7. CODE AND VIDEO DEMONSTRATION LINK

Code: <https://drive.google.com/drive/folders/1wHgl6LkvLGSLJWK4zlebe5ySidZe07WX?usp=sharing>

Video Demonstration: <https://youtu.be/OzS8yIfglXg?feature=shared>